AI2002 Artificial Intelligence - Phase 3 Report (Track A: R&D)

From Prediction to Prescription: An Explainable AI Framework for Value-Based Customer Retention in the Telecom Sector

Group F26-11 | Saqib Qazi, Hassan Shakil, Ammara Saleem | Code: github.com/HassanShakilPasha/telecom-value-based-retention (branch saqib-phase3) | Notebook: F26-11.ipynb

1. Scope of this phase

Phase 3 builds and runs the first layer of our framework, **churn prediction**, on the dataset prepared in Phase 2 (Maven Analytics Telecom Customer Churn: 7,043 customers, 26.5% churners). Three models were trained and compared on a validation set, the best one was evaluated once on a held-out test set, and its feature importances were inspected. CLV valuation, cost-sensitive learning, SHAP-based explanation and the budget optimizer belong to the next phases (Section 8).

2. Model / algorithm selection and justification

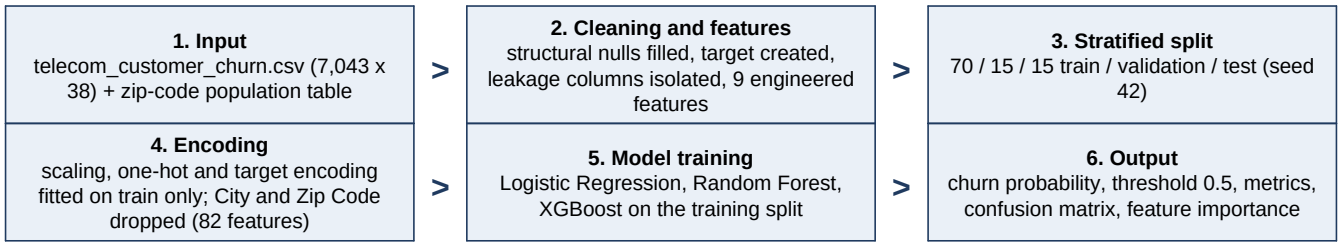
Model	Type	Why it was chosen
Logistic Regression	Linear baseline	Simple, fast and interpretable. Gives a reference score that any more complex model has to beat. Works well here because Phase 2 already scaled the numeric inputs.
Random Forest	Bagged decision trees	Captures non-linear effects and interactions (for example short tenure combined with a month-to-month contract) without manual feature crosses; robust to noise and needs little tuning.
XGBoost	Gradient-boosted trees	Typically among the strongest methods for small and medium tabular data; handles class imbalance directly (scale_pos_weight); and supports TreeSHAP, which our explainability layer needs in Phase 4.

Churners are the positive class (Churn = 1) and make up 26.5% of the data, so class weighting is used for all three models. Neural networks were not used in this phase: with about 4,900 training rows of tabular data, tree ensembles commonly match or beat them and are easier to explain with SHAP. They remain a possible extension.

3. Implementation details

Python 3 in a Jupyter notebook. **pandas** and **NumPy** for data handling, **scikit-learn** (LogisticRegression, RandomForestClassifier, evaluation metrics), **XGBoost** (XGBClassifier), **matplotlib** for plots and **tqdm** for training progress bars. The preprocessing code from Phase 2 lives in `src/` (loader.py, preprocessor.py) and is shared by the team; the modelling notebook is `Notebook/F26-11.ipynb`. A fixed random seed (42) is used everywhere, so results are reproducible.

4. Code workflow / pipeline (data input to model to output)



- **Cleaning (Phase 2).** Nulls that only mean 'service not subscribed' were filled with 'No Internet', 'No Phone', 'No Offer' or 0. Churn = 1 only for customers whose status is Churned (Stayed and Joined = 0). Churn Category and Churn Reason exist only after a customer leaves, so they are excluded as target leakage.
- **Feature engineering (Phase 2).** Nine added features: number of internet add-ons, age band, tenure group, has-dependents and has-referrals flags, and four charge ratios.
- **Encoding (Phase 2).** StandardScaler for numeric columns, one-hot encoding for low-cardinality categoricals and target encoding for City and Zip Code, all fitted on the training split only (84 encoded features). In Phase 3 City and Zip Code were dropped (Section 7), leaving 82 features.

5. Training process

Split	Rows	Churners	Churn rate	Use
Train	4,930	1,308	26.5%	Fit the models
Validation	1,056	280	26.5%	Compare models
Test	1,057	281	26.6%	Final, one-time evaluation

Model	Hyperparameters
Logistic Regression	max_iter = 2000, class_weight = balanced, L2 penalty (scikit-learn default), random_state = 42
Random Forest	n_estimators = 300, max_depth = None, min_samples_leaf = 5, class_weight = balanced, random_state = 42
XGBoost	n_estimators = 300 boosting rounds, learning_rate = 0.05, max_depth = 4, subsample = 0.8, colsample_bytree = 0.8, scale_pos_weight = 2.77 (non-churners / churners), eval_metric = logloss, random_state = 42

None of these models trains in epochs: Logistic Regression and Random Forest fit in a single call and XGBoost runs 300 boosting rounds. Class imbalance is handled with class weights (churners count about 2.8 times as much as non-churners). Hyperparameters were set by hand and not searched; the validation set was used only to compare the three models and the test set was used once, at the end. The decision threshold is the default 0.5, which Phase 4 replaces with a cost-based threshold. All models train in seconds on a laptop CPU, with tqdm progress bars.

6. Initial results

Validation set (1,056 customers, 280 churners).

Model	ROC-AUC	PR-AUC	Precision	Recall	F1	Accuracy
Logistic Regression	0.886	0.736	0.568	0.821	0.672	0.787
Random Forest	0.895	0.753	0.634	0.750	0.687	0.819
XGBoost	0.898	0.769	0.602	0.800	0.687	0.807

All three models separate churners from loyal customers well (ROC-AUC 0.886 to 0.898). XGBoost is best on ROC-AUC and PR-AUC, but the gaps are small and the validation set has only 280 churners, so we treat the models as comparable with XGBoost slightly ahead. The precision and recall differences reflect different operating points at the same 0.5 threshold: Logistic Regression flags the most customers (higher recall, lower precision) and Random Forest the fewest. Accuracy is shown for completeness only, because predicting 'stays' for everyone already scores 73.5%; PR-AUC and recall are the more informative measures here. XGBoost was selected for its ranking quality, especially among the highest-risk customers (left side of the precision-recall curve), which matters when a retention budget only covers a few customers.

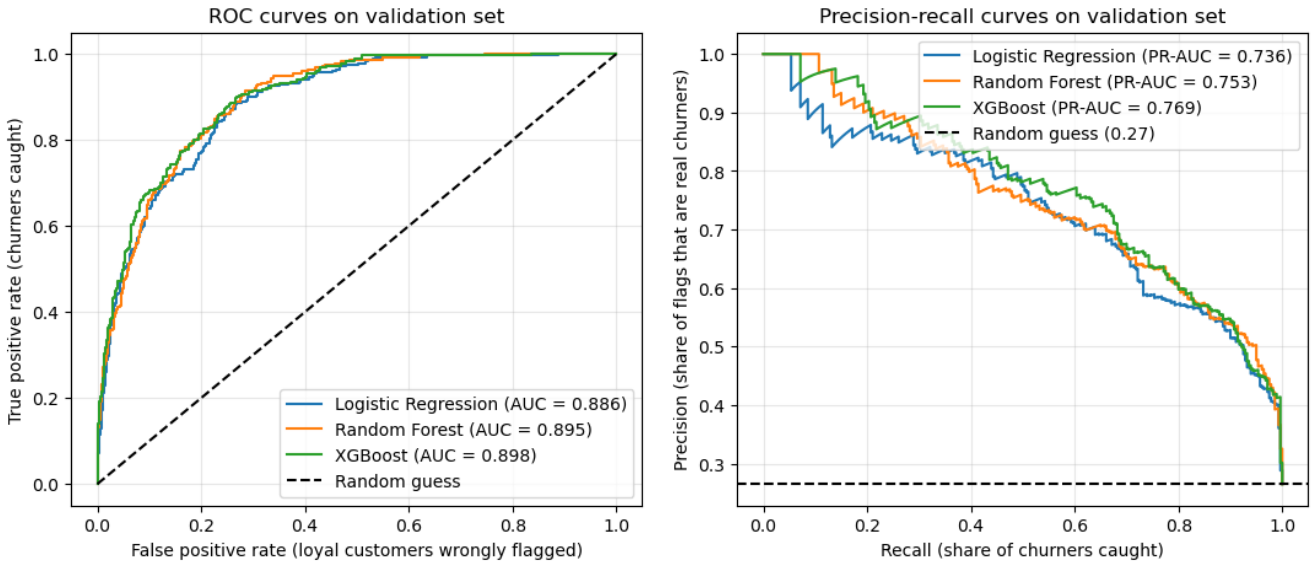


Figure 1. ROC (left) and precision-recall (right) curves of the three models on the validation set.

Test set (XGBoost, evaluated once).

	ROC-AUC	PR-AUC	Precision	Recall	F1	Accuracy
Validation	0.898	0.769	0.602	0.800	0.687	0.807
Test	0.908	0.788	0.604	0.836	0.701	0.811

On the test set the model catches 235 of 281 churners (recall 0.836) with 154 false alarms; the confusion matrix is [[622, 154], [46, 235]]. Test scores are in line with validation, which shows no sign of overfitting. The small improvement on test is within the noise expected from about 280 churners per set and should not be read as better generalisation.

Feature importance. Gain-based importance from XGBoost, with the one-hot columns summed back into their original features (Figure 2). Contract type accounts for 33% of the importance. Support add-ons (Premium Tech Support and Online Security) contribute about 10% together, and household features (marital status and dependents) about 10%. This agrees with the raw data: month-to-month customers churn at 45.8% against 2.5% on two-year contracts, and internet customers without Online Security churn at 41.8% against 14.6% with it. Importance shows what the model relies on, not what causes churn; the offers in the data were not randomised, so it cannot be read as the effect of changing a feature. Correlated columns (for example the tenure and revenue columns) share importance, so no single one stands out.

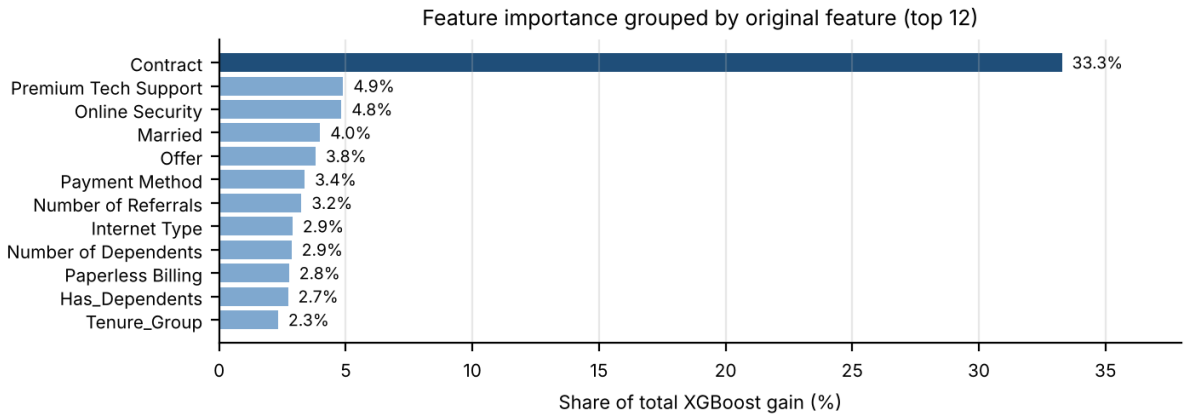


Figure 2. Share of XGBoost's total gain by original feature.

7. Challenges faced

- **Leakage through target encoding.** City and Zip Code were target-encoded in Phase 2, but the training rows were encoded with a fit-then-transform call that uses each customer's own label. Checked on the saved files, Zip Code alone scored ROC-AUC 0.87 on the training set but 0.54 on validation (City: 0.81 vs 0.55). With roughly three customers per zip code the encoding memorised the labels. Both columns were dropped for modelling; the shared Phase 2 preprocessing code should be corrected (cross-fitted encoding) before they are reused.
- **XGBoost feature names.** XGBoost rejected the one-hot column Age_Band_<30 because names may not contain '<', '[' or ']'. The columns were renamed (Age_Band_lt_30) in all three splits.
- **Class imbalance and the threshold.** With 26.5% churners, accuracy is misleading and the 0.5 threshold gives different precision/recall balances for each model. We use class weights and report PR-AUC and recall; the threshold will be set from costs in Phase 4.
- **Data limitations carried forward.** 454 'Joined' customers (very new, outcome unknown) are labelled as not churned; 120 rows have a negative Monthly Charge that has not been treated yet and will matter for CLV; and there is no intervention outcome data, so retention-offer effectiveness has to be assumed and tested by sensitivity analysis.

8. Next steps

Phase 4 onwards: compute customer lifetime value (12-month, survival-adjusted); add cost-sensitive learning (CLV-weighted training and calibrated probabilities); replace the 0.5 threshold with the rule 'expected saved value exceeds intervention cost'; compute SHAP values grouped by actionable driver; build the budget-constrained optimizer; and add the fairness audit and Power BI dashboard.